

PRD — Hyperliquid 大戶監控與警報 Dashboard

Sep 29, 2026 · @Paul

1. 背景與目標

第1版是一個自用的 Hyperliquid 大戶監控 dashboard：持續追蹤 CopyDog top100 地址的成交與倉位，把有資訊量的動作以 Telegram 通知，並把 Hyperliquid 不保留的歷史自己存下來。跟單執行不在本版範圍。

要解決的問題

- Hyperliquid 鏈上資料公開，但沒有現成工具能同時監看 100 個地址並在開倉、翻倉當下推送通知。
- Hyperliquid 只保留有限的成交歷史，沒有權益曲線；不從今天開始存，之後做績效評估與回測都沒有資料。
- 單一大戶的訊號雜訊高；100 個地址的群體行為（共識、淨偏向、集體離場）只有自建系統才看得到。

目標（可驗證）

1. 任一被監控地址開倉或翻倉後 5 秒內收到 Telegram 通知。
2. 100 個地址的成交、倉位、權益從上線第一天起完整落庫，零漏筆（以每小時對帳為準）。
3. 群體規則（共識、淨偏向翻轉、集體離場）可設定並觸發通知。
4. 每則通知有日誌，可回看通知後 1h / 4h / 24h 的價格變化。

非目標

- 不執行跟單、不簽任何交易、不保管任何私鑰。
- 不做多使用者、不做登入系統（單一 env token）。
- 不自建全網 leaderboard 索引；地址池來自 CopyDog 匯出與手動加入。
- 不做行動裝置 App；Telegram 即行動端。

2. 使用者與使用情境

唯一使用者是 Paul 本人，在電腦上看 dashboard、在手機上收 Telegram。

情境	觸發	使用者要做的事	系統要給的
即時決策	Telegram 通知：某地址開倉 / 翻倉	30 秒內判斷要不要跟進	地址標籤、幣種、方向、名目、槓桿、均價、該地址近 30 天勝率、dashboard 連結
每日巡檢	早上打開 dashboard	看昨晚誰動了、哪些幣被集體看多/看空	Live Feed、幣種 Heatmap、名單排名變化
研究單一大戶	收到多次同一地址通知	決定調高或調低其關注等級	倉位、成交歷史、自存權益曲線、幣種分布
規則調校	每週	淘汰噪音規則	警報日誌 + 通知後 1h/4h/24h 價格變化
名單更新	CopyDog 每次匯出	上傳 CSV	新進榜 / 掉榜 / 排名變化 diff

3. 系統本質與設計原則

系統本質是一條管線：地址池 → 事件流 → 狀態快照 → 規則 → 通知。所有功能都落在這五段之一。

1. **存下 Hyperliquid 不保留的東西。** fills 與 equity_snapshots 從第一天寫入，永不刪除；這是系統長期價值所在。
2. **WS 給即時，輪詢給正確。** 成交靠 WebSocket 推送；倉位與權益靠定時輪詢 clearinghouseState 對帳，補 WS 漏訊。
3. **先聚合再判斷。** 一筆大單會拆成多個 fill；規則只看聚合後的「動作」，不看單一 fill，避免洗版與誤判。
4. **每則通知可追溯、可評分。** 沒有日誌的規則不能上線。
5. **鏈無關的資料模型。** 核心表帶 chain 欄位，之後 Solana 聰明錢包目錄共用同一套後端與規則引擎。
6. **最小基礎設施。** 一個 Nest 進程、一個 Postgres，沒有 Redis、沒有獨立 worker；需要時再拆。

4. 功能需求

優先級：P0 = 第 1 版必須，P1 = 第 1 版應有，P2 = 之後。

4.1 地址池管理

ID	需求	優先級	驗收
A1	上傳 CopyDog top100 CSV/JSON；欄位對應（address、rank、其餘欄位存 stats_json）；每次匯入存成一個版本	P0	匯入後 leader_list_items 有 100 筆，帶 list_id 與 rank
A2	匯入時自動去重、把新地址加入 leaders 並啟動監聽	P0	匯入後 60 秒內新地址開始收到 fills
A3	手動新增/停用地址、標籤、備註、關注等級（A/B/C，決定通知強度）	P0	停用後不再產生通知與快照
A4	名單 diff：兩個版本之間的新進榜、掉榜、排名升降	P1	Leaders 頁可切換任兩版比較
A5	匯入時回補該地址可取得的歷史成交（userFillsByTime 拉到極限）	P0	匯入後 fills 表有該地址的可用歷史

4.2 Watcher

ID	需求	優先級	驗收
W1	對每個 active 地址訂閱 WS userFills；連線斷開自動重連並恢復全部訂閱	P0	手動斷網 30 秒後所有訂閱恢復，期間漏掉的 fill 由 W4 補齊
W2	fill 以 tid 去重後寫入 fills 表	P0	同一 tid 重送不產生重複列
W3	聚合窗：同地址、同幣、同方向的 fills 在 1 秒內合併為一個 action；以 fill 的 dir 欄位分類為 open / add / reduce / close / flip	P0	一筆拆成 8 個 fill 的大單只產生 1 個 action
W4	定時輪詢 clearinghouseState（每地址每 5 分鐘）寫 position_snapshots 與 equity_snapshots；同時對帳，發現倉位變化但無對應 fill 時補拉 userFillsByTime	P0	每地址每小時至少 12 筆 equity_snapshots
W5	訂閱 WS userEvents 取得清算事件	P1	清算事件寫入 actions（kind = liquidation）

ID	需求	優先級	驗收
W6	請求預算：info API 呼叫排程分散，總 weight 不超過 Hyperliquid 每分鐘 IP 上限的 70%	P0	連續 24 小時無 429

4.3 警報規則

每條規則共同欄位：啟用/停用、冷卻時間（同規則同地址同幣種內不重複通知）、適用關注等級（不設靜音時段，音量由關注等級控制）。

ID	規則	範圍	參數	優先級
R1	開新倉	單地址	50,000 USD 或該地址權益 10%，取低	P0
R2	翻倉（多轉空或空轉多）	單地址	無	P0
R3	大額動作：名目 $\geq X$ USD 或 $\geq$ 該地址權益 Y%	單地址	$X = 100,000$ USD、 $Y = 20\%$ （初始值，依日誌調整）	P0
R4	進入該地址從未交易過的幣種	單地址	回看 90 天	P1
R5	被清算、或權益自 24h 高點回撤 $\geq Z\%$	單地址	Z	P1
R6	共識：N 個地址在 T 分鐘內對同一幣同方向開倉	群體	N、T	P1
R7	淨偏向翻轉：某幣 top100 多空名目比跨越 1.0	群體	觀察窗	P1
R8	集體離場：某幣持倉地址數在 T 分鐘內下降 $\geq N$	群體	N、T	P2
R9	名單變動：新進榜地址首次開倉	群體	無	P2

4.4 通知

ID	需求	優先級
N1	Telegram bot 推送到兩個 chat（即時：R1-R5；群體：R6-R9）；訊息含地址標籤、動作、幣種、方向、名目、槓桿、均價、該地址近 30 天勝率、dashboard 詳情連結	P0
N2	每則通知寫 alerts 表（rule_id、address、coin、payload、sent_at）	P0
N3	通知評分：排程在 1h / 4h / 24h 後記錄該幣中間價，寫回 alerts	P1
N4	發送失敗重試 3 次，仍失敗記錄狀態	P0

4.5 Dashboard 頁面

ID	頁面	內容	優先級
D1	Live Feed	全地址 actions 時間軸，篩選：幣種、動作類型、關注等級；每列可展開看組成的 fills	P0
D2	Leaders	表格：CopyDog 排名、標籤、關注等級、當前倉位數、近 7/30 天已實現 PnL、勝率、平均持倉時間、最後動作時間；排序與篩選	P0
D3	Leader 詳情	當前倉位表、成交歷史、自存權益曲線、幣種分布、該地址觸發過的警報	P0
D4	Heatmap	幣種 × 多空名目、持倉地址數、24h 變化	P1
D5	Alerts	規則列表與編輯、警報日誌、每條規則的評分統計（通知後 24h 平均價格變化）	P0 規則 / P1 評分
D6	Lists	匯入介面、版本列表、diff 視圖	P0 匯入 / P1 diff

視覺：Next.js + shadcn，以 ReUI 元件庫為主，深色主題，密度參考 CopyDog。

5. 資料來源與 Hyperliquid API 約束

只用 Hyperliquid 公開 API，不用付費資料商。

需求	端點	用法	已知約束
即時成交	WS <code>userFills</code>	每地址一個訂閱	單一連線訂閱數有上限，100 個在限額內； <code>dir</code> 欄位直接給出 Open Long / Close Short / Long > Short 等分類
清算	WS <code>userEvents</code>	每地址一個訂閱	事件型別需在實作時對照官方 schema
倉位與權益	POST info <code>clearinghouseState</code>	每地址每 5 分鐘	按 IP 計 weight，每分鐘有上限；100 地址 × 每 5 分鐘 = 每分鐘 20 次，在上限內
歷史回補	POST info <code>userFillsByTime</code>	匯入時與對帳補拉	只回最近一段區間、單次筆數有上限；無法回補更早歷史
幣種資訊	POST info <code>meta</code>	啟動時與每日	szDecimals、最大槓桿
中間價（通知評分用）	POST info <code>allMids</code>	排程	一次回所有幣
名單	CopyDog 匯出檔	手動上傳	欄位格式以實際匯出檔為準，匯入器需可調對應
名單交叉驗證	Hyperliquid 官方 leaderboard JSON	P2，手動觸發	非正式端點，格式可能變動

實作前確認清單：

- ☐ 實測 WS 每連線訂閱上限與每 IP 連線上限
- ☐ 實測 info API 每分鐘 weight 上限，據此定 W6 的預算
- ☐ 實測 `userFillsByTime` 可回溯的最遠時間
- ☐ 取得一份 CopyDog 匯出檔，定匯入器欄位對應

6. 資料模型

Postgres，所有核心表帶 `chain` 欄位（第 1 版恆為 `hyperliquid`）。`fills` 與 `equity_snapshots` 只增不刪。

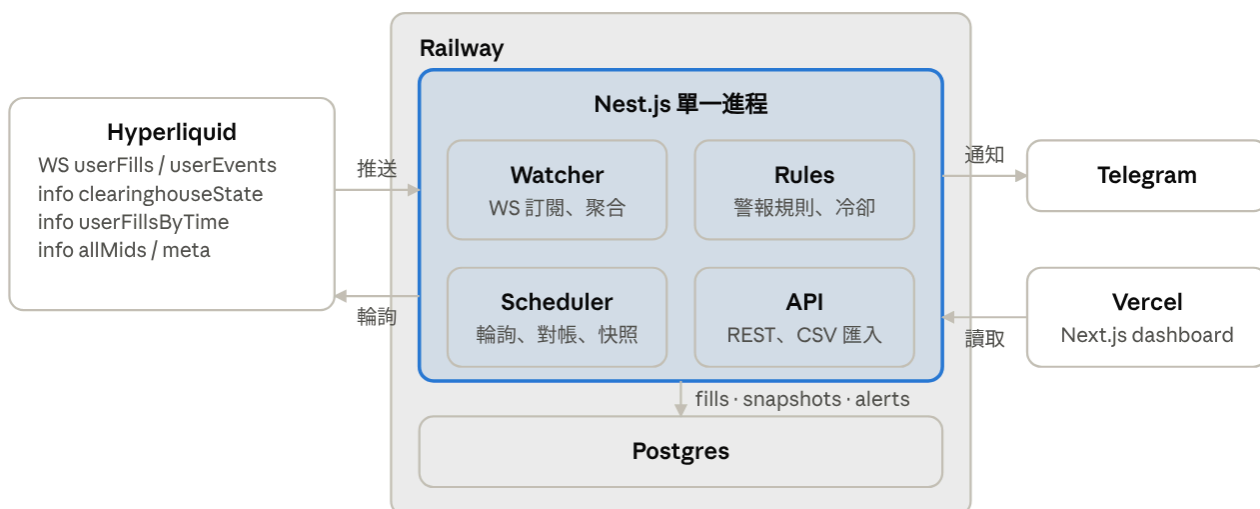
表	主鍵	主要欄位	用途
leader_lists	id	source, imported_at, file_name	每次匯入一個版本
leader_list_items	(list_id, address)	rank, stats_json	名單版本內容, diff 用
leaders	(chain, address)	label, tier(A/B/C), notes, active, first_seen_at	地址池主檔
fills	(chain, tid)	address, coin, side, dir, px, sz, fee, closed_pnl, hash, ts	原始成交, 永久保存
actions	id	chain, address, coin, kind(open/add/reduce/close/flip/liquidation), side, notional_usd, avg_px, leverage, fill_ids[], ts	聚合後的動作; 規則與 Feed 只看這張表
position_snapshots	(chain, address, coin, ts)	sz, entry_px, leverage, margin_mode, unrealized_pnl, liq_px	定時倉位快照
equity_snapshots	(chain, address, ts)	account_value, total_margin_used, withdrawable	權益曲線
coin_meta	(chain, coin)	sz_decimals, max_leverage, updated_at	精度與槓桿
alert_rules	id	scope(address/group), kind(R1-R9), params_json, cooldown_s, quiet_hours, tiers[], enabled	規則
alerts	id	rule_id, chain, address?, coin?, action_id?, payload_json, sent_at, send_status, px_at_send, px_1h, px_4h, px_24h	日誌與評分

索引：fills(address, ts desc)、actions(ts desc)、actions(coin, ts desc)、equity_snapshots(address, ts desc)。

估算容量：100 地址、平均每地址每日 50 fills → 每年約 180 萬列 fills，Postgres 單表無壓力。

7. 系統架構與部署

一個 Nest 進程做完監聽、輪詢、規則與 API，Vercel 只放前端



系統架構 · 2 個平台、1 個進程、1 個 DB

Watcher 與 Scheduler 寫入 Postgres，Rules 讀 actions 表決定是否推 Telegram，Next.js 只透過 REST 讀資料與上傳 CSV。

元件	部署	說明
Next.js (TS) + shadcn / ReUI	Vercel	純前端；以 env 內的 token 呼叫 API
Nest.js (TS)	Railway, 1 個服務, 關閉 sleep	模組：Watcher、Scheduler (@nestjs/schedule)、Rules、Notify、API、Import；共用一個進程，避免 WS 訂閱與規則狀態跨進程同步
Postgres	Railway	每日自動備份；fills 與 equity_snapshots 為主要資產
Telegram Bot	—	一個 bot token、一個 chat id，存於 Railway env

區域：Hyperliquid 節點在東京，Railway 選新加坡區域，延遲約 70 ms，對通知用途足夠。

擴展點（不在第 1 版做）：Watcher 拆成獨立服務並用 Redis 傳 actions；地址數超過單一 WS 連線上限時分連線；跟單執行加在 Rules 之後作為另一種 sink。

8. 非功能需求

面向	要求	做法
延遲	fill 抵達到 Telegram 送出 ≤ 5 秒（含 1 秒聚合窗）	規則在記憶體內評估，不經佇列
完整性	fills 零漏筆	WS 去重 + 每 5 分鐘倉位對帳 + 差異時補拉 userFillsByTime
可用性	Railway 服務重啟後 60 秒內恢復全部訂閱	訂閱清單來自 DB，啟動即重建；不依賴進程內狀態
限速	24 小時內 0 次 429	集中式請求排程器，總 weight 控制在上限 70%
可觀測	知道 Watcher 是否活著	心跳表：每分鐘寫最後收到 fill 時間、WS 狀態、當日請求數；dashboard 頂部顯示；超過 10 分鐘無 fill 且 WS 斷線 → Telegram 自我告警
安全	不持有任何私鑰；API 只有本人能打	前端以 env token 呼叫，Nest 以 guard 驗證；Telegram 只推不收指令
資料保存	fills、equity_snapshots 永久；position_snapshots 保留 180 天	Railway Postgres 每日備份，另每週 pg_dump 到本機
成本	每月 ≤ 20 USD	Vercel 免費層 + Railway 一個服務一個 DB

9. 里程碑與交付順序

三週到最小可用與群體訊號，之後持續調規則



里程碑 · 4 個階段、3 個閘門

每個閘門都是真實資料上的驗收，不是功能完成；M1 先上線是為了讓 fills 與 equity_snapshots 盡早開始累積。

順序原則：

1. Engine 先於 UI。M1 的 dashboard 只需要一個匯入頁，其餘看 Telegram。
2. 落庫先於規則。規則可以事後重算，漏掉的 fill 回不來。
3. 單地址規則先於群體規則。群體規則需要穩定的 actions 表與幾天的資料才能驗證。

10. 風險與未決問題

風險	影響	對策
Hyperliquid 調整 WS 訂閱上限或 info 限速	Watcher 失效或大量 429	W6 預算可調；訂閱數超限時分多條連線（擴展點已預留）
CopyDog 名單本身有偏差（只列近期高 ROI、倖存者偏差）	跟到運氣好的地址	名單版本化 + 自算 30 天指標，用自己的數據覆蓋 CopyDog 排名；P2 交叉比對官方 leaderboard
大戶用子帳號或 vault 分散交易	只看到部分倉位	第 1 版接受；Leader 詳情頁標示該地址是否為 vault
群體規則在 100 個地址下樣本太小，共識訊號偶發	誤報	先以 P1 觀察一個月，用 N3 評分決定是否保留
Railway 單服務單點	重啟期間漏訊	對帳補拉覆蓋 5 分鐘內的漏訊；心跳告警讓本人及時知道
通知過多變成噪音	忽略真正訊號	冷卻時間 + 關注等級 + 靜音時段從 M1 就上；M4 靠評分淘汰

未決問題：

- ☐ CopyDog 匯出檔的實際欄位（決定匯入器與 stats_json 內容）— 待樣本檔
- ☐ 專案名稱與 repo 名

11. 決策紀錄

2026-09-29 拍板，以下為第 1 版的固定決定；改動需回到本表。

主題	決定	理由
聚合第二層	actions 保留 1 秒窗；另建 episodes 視圖：同地址同幣同方向 15 分鐘內累計，通知附本次累計名目	大戶分批建倉是常態，1 秒窗會把一次建倉拆成多則通知

主題	決定	理由
關注等級初始值	全部 B；CopyDog 前 20 名為 A；一週後依評分調整	先有基準再分級
R1 門檻	50,000 USD 或該地址權益 10%，取低	純絕對值會漏掉小資金高勝率地址
市場範圍	只看 perps，不看 spot	spot 訊號品質低，schema 不同
fills 原始資料	加 <code>raw jsonb</code> 欄位保存原始 fill	schema 變動時可重算
Telegram 頻道	兩個 chat：即時（R1-R5）、群體（R6-R9）	群體訊號少但重要，不能被洗掉
靜音時段	不設；音量由關注等級控制	市場 24 小時，台北凌晨常是美股時段
通知輔助資訊	只放一個數字：該地址近 30 天在該幣的勝率；M2 起加入	需要 fills 累積後才有意義
ORM	Drizzle	Postgres 型別最直接，migration 為純 SQL
Repo 結構	pnpm workspace + Turborepo monorepo； <code>packages/shared</code> 放 zod schema 與型別	Vercel 與 Railway 皆支援子目錄部署
Hyperliquid client	自行封裝 WS 與 info 兩個唯讀 client	現成 SDK 為下單設計；限速與重連要自己掌握
前端即時更新	React Query 每 10 秒輪詢 REST；不做 WS 到前端	即時性由 Telegram 提供，dashboard 是回看用
測試環境	直接接 mainnet 唯讀； <code>DRY_RUN=true</code> 時通知只寫日誌	testnet 大戶資料無意義
時區	DB 全 UTC；前端與通知顯示 Asia/Taipei	—
<code>chain</code> 欄位	第 1 版即加，但不做任何抽象層	為 Solana 預留，避免過度設計
名單更新	每週手動匯出 CopyDog 一次；觀察變動率後再決定是否自動化	先量再投資
權益曲線顯示	儲存維持 5 分鐘一筆；詳情頁預設每小時一點，可切 5 分鐘	儲存不降採樣，顯示才降
勝率與 PnL 單位	以一次完整倉位（開到平）為一筆；直接加總平倉 fill 的 <code>closed_pnl</code>	Hyperliquid 已算好，不需自行配對
R4 回看天數	90 天；上線初期等同「有資料以來」	受限於自存歷史

主題	決定	理由
群體規則初始值	R6：N=3、T=30 分鐘；R7：觀察窗 24 小時；R8：N=3、T=60 分鐘	先偏鬆，看誤報再收緊
冷卻時間預設	單地址規則 15 分鐘（同地址同幣）；群體規則 4 小時（同幣）	避免洗版
Vault 地址	匯入時以 clearinghouseState 判斷並標記；照常監控；R3 的權益 % 改用 vault TVL	名單可能混有 vault